
第 18 章

图像处理大作业

MATLAB 高级编程与工程应用

本章目录

- §18.1 图像处理基础：数字图像与彩色图像
- §18.2 JPEG 图像压缩编码
- §18.3 信息隐藏
- §18.4 人脸检测（基于颜色直方图）

本章环境卡（系统提示词）

用户正在学习信号与系统和MATLAB仿真，以下要求适用于本次会话里所有 MATLAB 代码请求：

- 兼容 MATLAB R2023b，不使用之后版本独有的函数；
- 只返回完整、可直接运行的单文件代码 + 必要的中文注释，代码之外不附任何解释文字；
- 变量名沿用我每次指定的名称，不要改名；
- 显示符号表达式时用 `disp`，不要用 `pretty`；
- 绘图规范：
 - 字号：图中所有文字（坐标轴刻度、`xlabel`/`ylabel`、`title`、`legend`、`text` 标注）统一 24 pt，不留默认小字号；
 - 线宽：所有曲线线宽 2；
 - 配色：多条曲线用深色高对比（蓝、红、黑、深绿），并兼用实线/虚线/点线区分，避免浅色（纯绿/青/黄）；
 - 网格：`grid on`；

本章环境卡（系统提示词）

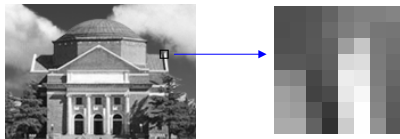
- 图例：仅当同一坐标轴内画有多条曲线时才加 `legend` 区分；单曲线图（含每个只画一条曲线的子图）不加 `legend`，其含义由标题与纵轴标签标明；
- 画布：单幅图用默认尺寸即可；纵向排列的多子图必须按子图数 `N` 加高、避免过扁，例如先 `figure('Position',[100 100 900 350*N])` 再 `subplot`；
- 图像显示：用 `imshow` 显示图像（按需 `axis image / axis off`）、多图对照用 `tiledlayout` 并给每张子图加 24 pt 标题；上面的网格/图例/线宽/配色规范只对曲线图（如直方图）适用，图像本身不加网格与图例；
- 不要添加任何测试桩、自检打印或调试残留。

本章导引：变换 + 取舍

- 图像处理与识别是应用篇的综合大作业，贯穿三条主题：**JPEG 压缩**、**信息隐藏**、**人脸检测**。它们都建立在第 17 章的一个事实上：图像的信息高度不均匀：能量集中在低频，人眼对高频细节不敏感，真实图像有大量冗余。
- 三项技术是同一方法的三种应用：**先变换、再取舍**。**压缩** = 把能量集中 (DCT)，再丢掉人眼不在意的高频 (量化)；**隐藏** = 把秘密放进人眼和压缩都不注意的地方 (最低位 / 中频系数)；**识别** = 用几个有判别力的特征概括整幅图 (颜色直方图)，用内积比较相似度。
- 与音乐、语音两个大作业一样走到 **L3 (目标委托)**：只描述效果与验收标准，实现全部交给模型，自己充当验收者。
- 用以核对的判据都是图像里可以人工计算的量：**维度**、**灰度级**、**PSNR**、**误码率**、**颜色直方图归一化内积**，都独立于实现。

§18.1 图像为什么要压缩

- 上一章已确认图像就是矩阵：灰度图是 $M \times N$ 的 uint8 矩阵、彩色图是 R、G、B 三层。一幅 1600×1200 的彩色照片，每像素三个分量各 8 比特，合计约 46 兆比特、5.8 MB，数据量很大，存储与传输都需要先压缩。
- 压缩就是用更少的数据表示同一幅图。图像允许有损：只要人眼难以察觉，丢掉部分信息也无妨。关键在于如何取舍。
- 本节先用两个例子看两种取舍：例 18.1 降低每个像素的幅度精度（量化）；例 18.2 把图像分成亮度与色度、只在人眼不敏感的色度上大幅取舍。§18.2 的 JPEG 再把这些取舍系统地组织起来。



数字图像即像素灰度矩阵

例 18.1 幅度量化（灰度级离散化）

把一幅灰度图的幅度量化为 4 个等间隔灰度级，与原图并排，观察幅度离散化（色阶／分层）。

请读入同目录下的清华二校门灰度照片 `thu_gate.jpg`，把它的幅度（灰度）量化为 4 个等间隔灰度级，
与原图并排显示，观察幅度离散化（色阶／分层）的现象。把量化结果命名为 `Iq`。

例 18.1 运行结果 · 原图与 4 级量化图

原图



4级量化图像



例 18.1 核对

对结果的核验

- 量化后整幅图的不同灰度取值个数应恰为 4，可用 `numel(unique(Iq))` 核对，其值应为 4，取值为 $\{0, \frac{1}{3}, \frac{2}{3}, 1\}$ ，远少于原图的数百级；取值仍落在 $[0, 1]$ 。
- 量化图应出现明显的色阶 / 分层，天空等平滑过渡处变成几片纯色台阶，级数越少越明显。

对代码的核验

- 量化用就近取整：`round(Id*(L-1))/(L-1)` 把 $[0, 1]$ 等分为 L 级；先 `im2double` 转 `double` 再算。这一步独立于模型的具体实现，可以人工核算。

§18.1 YCbCr: 把亮度和颜色分开

- RGB 三个通道各自都混着明暗与颜色：某处偏红，则 R 大、 G 与 B 小，明暗与色彩混在一处，不便分别取舍。
- **YCbCr** 把二者分开：亮度 Y 描述明暗与结构，色度 C_b 、 C_r 描述颜色，都由 RGB 线性组合得到：

$$\begin{aligned}Y &= 0.299R + 0.587G + 0.114B, \\C_b &= 128 - 0.169R - 0.331G + 0.5B, \\C_r &= 128 + 0.5R - 0.419G - 0.08B.\end{aligned}$$

MATLAB 用 `rgb2ycbcr` / `ycbcr2rgb` 互转。

- 人眼对亮度的空间细节很敏感，结构稍一模糊即可察觉；对色度的空间细节却不敏感。因此可对色度大幅取舍、对亮度精细保留。下一例验证这一点。

例 18.2 亮度与色度的不同取舍

把彩色图转到 YCbCr，对色度、亮度分别做同样粒度的降采样，再在 RGB 上做对照，观察人眼对二者的不同敏感度。

请读入同目录下的清华大礼堂彩色照片 `thu_auditorium.jpg`，转到 YCbCr 空间。取舍方式统一为：把某个分量每 8x8 块只保留平均值（块内细节全部丢弃）。分三种情形各做一遍：

- (1) 只对色度 Cb、Cr 这样做，亮度 Y 保持不变；
- (2) 只对亮度 Y 这样做，色度保持不变；
- (3) 直接在 RGB 上只对 R 通道这样做，G、B 保持不变。

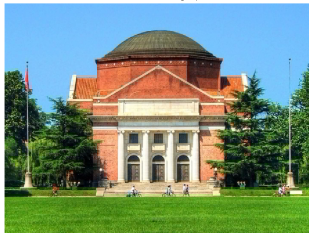
把原图与三种结果排成 2x2 显示对比，三种结果分别命名为 `imgChroma`、`imgLuma`、`imgRGB`。

例 18.2 运行结果 · 三种取舍对照

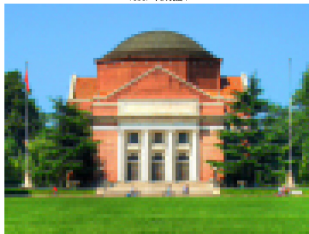
原图



YCbCr: 只降色度 C_b 、 C_r



YCbCr: 只降亮度 Y



RGB: 只降 R 通道



例 18.2 核对与易错点

对结果的核验

- 只对色度取平均，结果与原图几乎无差别，结构、砖纹清晰；只对亮度取平均，出现明显的 8×8 马赛克。可见亮度承载结构、人眼敏感，色度则可大幅粗化而人眼难以察觉。
- 在 **RGB** 上只对 R 通道取平均，**红色区域**（红砖、红旗、红屋顶）明显糊掉、蓝天绿树基本不变，因为 R 通道混着红色物体的明暗结构。同样只动一部分数据，YCbCr 降色度看不出、RGB 降 R 却伤了结构。可见只有先转 YCbCr、把结构集中到亮度 Y 单独保留，才能只对色度取舍而人眼难以察觉。这正是 JPEG 处理彩色图时先转 YCbCr、再把色度按更低分辨率保存、亮度保留精细的原因。

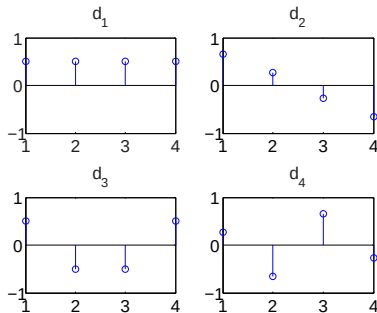
§18.2 JPEG 压缩：信源编码与有损压缩

- 把源数据编成更短的码字以便存储与传输，称为**信源编码**，与语音编码、视频编码同属一类。§18.1 已见幅度量化与色度取舍两种手段，JPEG 把它们系统地组织成一套完整流程。
- **无损编码**（如 Huffman、Zip）解码后与原数据完全相同，但压缩比有限，文本一般只有 3 ~ 5 倍。图像、语音数据量大、又可容忍失真，改用**有损编码**：在一定失真容限下大幅提高压缩比，只要不影响主观感受即可。
- JPEG 是典型的有损编码。本节讲它的核心：灰度图按 8×8 块做 **DCT**、用量化丢掉人眼不敏感的高频、再经之字形扫描与熵编码；解码后用 **PSNR** 客观评价质量。

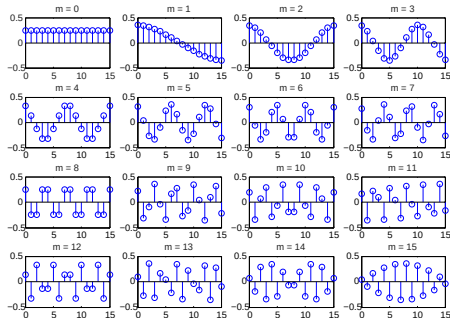
§18.2 DCT (一): 为什么只用余弦

- 第 9、17 章的傅里叶变换用复指数把信号分解成正弦与余弦，系数是复数。对实数图像，为避免复数，只保留余弦、丢掉正弦，就得到离散余弦变换 (DCT): 系数是实数、计算更省。MATLAB 用 `dct2` / `idct2` 做二维 DCT 与逆变换。
- 长度 N 的信号 $\mathbf{p} = [p_0, \dots, p_{N-1}]^T$ ，其 DCT 系数 $\mathbf{c}$ 写成矩阵即 $\mathbf{c} = \mathbf{D}\mathbf{p}$ 。 $\mathbf{D}$ 是 $N \times N$ 的正交矩阵，故逆变换 (IDCT) 是 $\mathbf{p} = \mathbf{D}^T \mathbf{c}$ 。
- 把 $\mathbf{D}^T$ 的各列记作基 $\mathbf{d}_0, \dots, \mathbf{d}_{N-1}$ ，则 $\mathbf{p} = \sum_k c_k \mathbf{d}_k$: 信号是这组余弦基的加权和，系数 c_k 是第 k 个基的权重。这与傅里叶级数把信号写成众多正弦之和是同一思路。

§18.2 DCT (二): 从平缓到振荡的余弦基



$N = 4$ 的四个一维 DCT 基

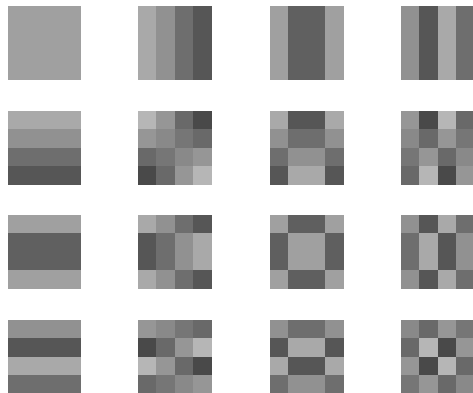


$N = 16$ 的十六个一维 DCT 基

- 第 0 个基是常数，对应直流（整体亮度）；序号越大，余弦振荡越快、频率越高。
- 任何信号都是这些基的加权和：低频基描述整体明暗，高频基描述细节纹理。某个基的权重越大，信号就含越多那种成分。

§18.2 DCT (三): 二维 DCT 与能量集中

- 图像是二维的，就在两个方向各做一次一维 DCT (先逐列、再逐行): $\mathbf{C} = \mathbf{D}\mathbf{P}\mathbf{D}^T$, 逆变换 $\mathbf{P} = \mathbf{D}^T\mathbf{C}\mathbf{D}$ 。二维基由一维基组合而成 (右图 $N=4$): 左上平缓、右下细密。
- 系数位置有明确含义: 左上角 $C_{0,0}$ 是直流 (DC), 即块的平均亮度; 其余为交流 (AC), 越往右下、对应横竖纹理变化越快、频率越高。
- 能量集中: 常见景物以缓慢变化为主, DCT 系数左上大、右下小, 右下角大量系数接近零。

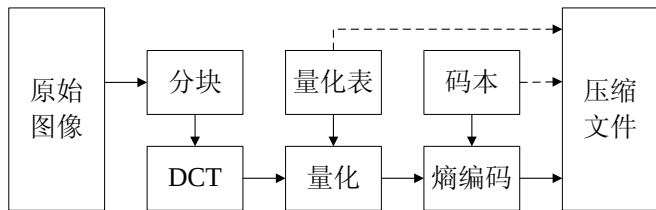


$N=4$ 的二维 DCT 基

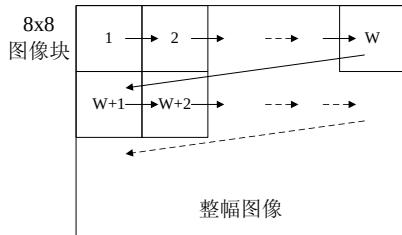
DCT 把 64 个像素变成 64 个系数、本身并不压缩; 但这 64 个系数不必等量看待, 右下角接近零的小数最先被丢弃。DCT 只是为压缩作准备, 真正减少数据的是下一步的量化。

§18.2 JPEG 编码流水线

灰度图先按 8×8 分块、每块像素减 128（把灰度移到零附近）；逐块做二维 DCT；用量化表把系数逐元素相除、四舍五入（高频步长大、大量归零，这一步是有损与失真的来源）；再之字形扫成一维、做熵编码（无损，本课不要求实现）。



JPEG 编码器结构



8×8 分块与扫描顺序

§18.2 一个 8×8 块的编码 (上): DCT 使能量集中

取一个有纹理的 8×8 灰度块, 做二维 DCT。原来 64 个灰度值分布均匀, 变换后能量集中到左上角, 右下多为小数 (系数已四舍五入到整数便于观察)。

79	80	85	99	121	128	128	132		1035	-122	-86	277	-39	-134	35	11
78	81	87	106	125	134	118	117		-128	-55	-8	-159	44	76	-7	-14
80	81	88	97	158	198	121	113		-25	28	58	-41	-26	27	-6	1
88	88	84	95	217	221	123	108	2D-DCT →	11	12	13	12	24	-10	-21	0
137	130	70	90	237	233	125	103		10	-7	-6	1	2	-5	-3	10
160	151	66	74	232	247	130	96		0	-17	-10	4	-5	-4	11	5
164	157	85	56	219	254	134	91		6	-4	5	8	-7	0	3	-6
171	171	98	48	204	254	146	87		5	2	5	4	-3	-1	4	-4

§18.2 一个 8×8 块的编码 (下): 量化使高频归零

DCT 系数逐元素除以量化表、四舍五入。量化表左上小、右下大 (人眼对低频敏感、对高频不敏感), 右下角系数大量变成 0。量化即 $\text{round}(\text{系数}/\text{步长})$, 例如左上角 $\text{round}(1035/16) = 65$ 。

1035	-122	-86	277	-39	-134	35	11	16	11	10	16	24	40	51	61	65	-11	-9	17	-2	-3	1	0
-128	-55	-8	-159	44	76	-7	-14	12	12	14	19	26	58	60	55	-11	-5	-1	-8	2	1	0	0
-25	28	58	-41	-26	27	-6	1	14	13	16	24	40	57	69	56	-2	2	4	-2	-1	0	0	0
11	12	13	12	24	-10	-21	0	14	17	22	29	51	87	80	62	1	1	1	0	0	0	0	0
10	-7	-6	1	2	-5	-3	10	18	22	37	56	68	109	103	77	1	0	0	0	0	0	0	0
0	-17	-10	4	-5	-4	11	5	24	35	55	64	81	104	113	92	0	0	0	0	0	0	0	0
6	-4	5	8	-7	0	3	-6	49	64	78	87	103	121	120	101	0	0	0	0	0	0	0	0
5	2	5	4	-3	-1	4	-4	72	92	95	98	112	100	103	99	0	0	0	0	0	0	0	0
DCT 系数								量化表								量化后							

64 个系数只剩左上角十来个非零、其余全为 0。之字形扫描后末尾一长串零可极紧凑地编码, 这才是 JPEG 省空间的来源。

§18.2 熵编码（一）DC 系数：差分 + Huffman

相邻块的 DC（直流）很接近，先差分（只存与前一块的差）、再作 Huffman 编码。差值按大小分类别（Category）查表得码字，后接差值的二进制（负数取反码）。

差值	Category	码字
0	0	00
± 1	1	010
$\pm 2, \pm 3$	2	011
$\pm 4 \sim \pm 7$	3	100
$\pm 8 \sim \pm 15$	4	101
$\pm 16 \sim \pm 31$	5	110
$\pm 32 \sim \pm 63$	6	1110

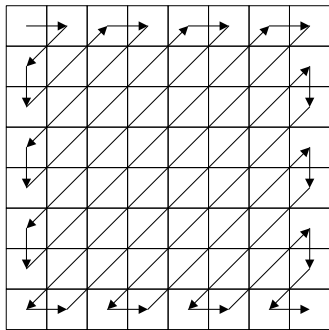
例：三个块 DC = [10, 8, 60]，差分得 [10, 2, -52]：

- 10 $\rightarrow$ Cat 4 (101) + 1010 = 1011010
- 2 $\rightarrow$ Cat 2 (011) + 10 = 01110
- -52 $\rightarrow$ Cat 6 (1110) + 反码 001011 = 1110001011

三段拼接即这三个块 DC 的码流。

§18.2 熵编码 (二) AC 系数: 之字形 + 游程

之字形把 8×8 系数排成一维 (低频在前、末尾一长串零)。游程编码把每个非零系数记成 (Run/Size, Amplitude): Run 是它前面零的个数、Size 由幅度定; 超过 15 连零用 ZRL, 末尾全零用 EOB。



之字形扫描

例: 某块之字形后为 $10, 3, 0, 0, 2, \underbrace{0 \dots 0}_{20}, 1, \underbrace{0 \dots 0}_{37}$

- 10: 前 0 零, (0/4) 码 $1011 + 1010$
- 3: (0/2) 码 $01 + 11$
- 2: 前 2 零, (2/2) 码 $11111000 + 10$
- 1: 前 20 零 = ZRL + (4/1) 码 $111011 + 1$
- 其后全零: EOB 码 1010

熵编码无损、不引入失真, 本课不要求实现。

例 18.3 8×8 分块 DCT + 量化 + Zig-Zag (编码器核心)

请读入同目录下的清华二校门灰度照片 `thu_gate.jpg`，实现 JPEG 编码器的核心：
先把像素整体减 128（把灰度移到零附近），再按 8×8 分块对每块做二维 DCT，用下面这张
JPEG 标准亮度量化表逐元素相除并四舍五入量化，最后取一个有纹理的块（如牌楼雕
饰处）的 8×8 量化块按之字形（低频在前）扫成 1×64 向量。量化表（直接采用）：

```
QTAB = [16 11 10 16 24 40 51 61
        12 12 14 19 26 58 60 55
        14 13 16 24 40 57 69 56
        14 17 22 29 51 87 80 62
        18 22 37 56 68 109 103 77
        24 35 55 64 81 104 113 92
        49 64 78 87 103 121 120 101
        72 92 95 98 112 100 103 99];
```

把整幅量化系数命名为 `Coeff`、单块量化结果命名为 `Cq`、之字形向量命名为 `zz`，并显示这个 1×64 的之字形向量 `zz`。

例 18.3 运行结果 · 之字形扫描向量 zz

运行输出如下（取牌楼一个有纹理的块，量化后只剩少数低频非零、末尾一长串零）：

有纹理块量化系数的之字形向量 zz：

列 1 至 13

65	-11	-11	-2	-5	-9	17	-1	2	1	1	1	4
----	-----	-----	----	----	----	----	----	---	---	---	---	---

列 14 至 26

-8	-2	-3	2	-2	1	0	0	0	0	0	0	-1
----	----	----	---	----	---	---	---	---	---	---	---	----

列 27 至 39

1	1	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---

列 40 至 52

0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---

列 53 至 64

0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---

例 18.3 核对与易错点

对结果的核验

- 单块量化结果 8×8 、非零系数远少于 64（高频除完归零，本例只剩左上角十来个低频非零）；DC（左上角）绝对值块内最大；zz 长 64、zz(1) 即 DC，末尾一长串零正好交给游程编码。
- 高频系数量化后归零，正是压缩与失真的来源，由非零系数的个数即可看出。

对代码的核验

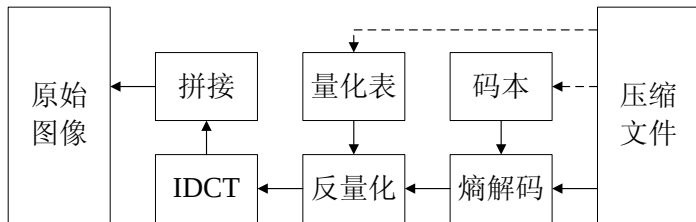
- 应先电平下移 -128、再逐 8×8 块（for 循环、按 1:8:end 取块）调 dct2、按 round(块./QTAB) 量化。

易错点：之字形须按列优先线性下标取

- 之字形索引 zz 须按 MATLAB 列优先的线性下标取（zz=Cq(ZZ)）；行/列优先混淆会打乱低频在前、高频在后的顺序。

§18.2 JPEG 解码：熵解码、反量化、逆 DCT

解码器与编码器结构相似、流程相反：熵解码恢复量化后的系数、反量化（乘回量化表）、二维逆 DCT、每像素加回 128，再拼回整幅图。



JPEG 解码器结构

反量化补不回被丢掉的信息。量化时右下角系数被大步长除、取整归零，反量化乘回步长也只能得到 0。恢复的系数与原系数存在误差，这正是重建图与原图不同（失真）的来源。

§18.2 一个 8×8 块的解码 (一): 反量化

熵解码恢复量化后的系数，再乘回量化表（编码时除以它、现在乘回来）。右下角被量化成 0 的高频乘回仍是 0，丢掉的信息补不回来。

$$\begin{array}{ccc} \begin{array}{cccccc} 65 & -11 & -9 & 17 & -2 & -3 & 1 & 0 \\ -11 & -5 & -1 & -8 & 2 & 1 & 0 & 0 \\ -2 & 2 & 4 & -2 & -1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} & \times & \begin{array}{cccccc} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{array} \\ \underbrace{\hspace{10em}}_{\text{量化后}} & & \underbrace{\hspace{10em}}_{\text{量化表}} \end{array} = \begin{array}{cccccc} 1040 & -121 & -90 & 272 & -48 & -120 & 51 & 0 \\ -132 & -60 & -14 & -152 & 52 & 58 & 0 & 0 \\ -28 & 28 & 64 & -48 & -40 & 0 & 0 & 0 \\ 14 & 17 & 22 & 0 & 0 & 0 & 0 & 0 \\ 18 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

$\underbrace{\hspace{10em}}_{\text{反量化系数}}$

§18.2 一个 8×8 块的解码 (二): 逆 DCT 与重建

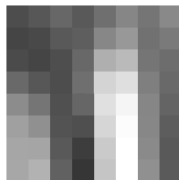
把反量化系数做二维逆 DCT、再加回 128, 得到重建块。与原块相比, 整体与主要纹理保留、细节有差异, 这就是有损压缩的失真。

重建块 =

77	87	103	89	111	134	117	137
69	73	89	94	135	152	113	122
75	69	78	104	175	186	117	109
105	87	77	111	212	223	127	105
140	117	80	102	224	245	134	100
160	144	83	79	214	252	135	93
166	165	89	61	202	256	138	89
167	178	97	54	197	261	144	90

原始块 =

79	80	85	99	121	128	128	132
78	81	87	106	125	134	118	117
80	81	88	97	158	198	121	113
88	88	84	95	217	221	123	108
137	130	70	90	237	233	125	103
160	151	66	74	232	247	130	96
164	157	85	56	219	254	134	91
171	171	98	48	204	254	146	87



重建块



原始块

§18.2 PSNR: 把重建误差写成分贝

- 有损压缩会引入误差，要有个客观数衡量重建图与原图差多少。先算均方误差 **MSE**: 两图逐像素差的平方的平均，越小越像。

$$\text{MSE} = \frac{1}{MN} \sum_{x,y} (\hat{f}(x,y) - f(x,y))^2.$$

- **PSNR** 把它换成分贝 (dB 已在第 5 章波特图中用过):

$$\text{PSNR} = 10 \lg \frac{255^2}{\text{MSE}} \text{ (dB)}, \quad \text{越大越好.}$$

经验上图像 $\text{PSNR} > 30 \text{ dB}$ 通常画质可接受。

- 量化越粗，误差越大、MSE 越大、PSNR 越低。量化的粗细，可以直接用 PSNR 量出来。

例 18.4 解码重建与 PSNR

实现 JPEG 解码（反量化、逆 DCT、加回 128），用 PSNR 客观评价重建质量；再改变量化的粗细重做，看画质如何随量化变化。

接着上一步：请对量化后的系数做 JPEG 解码——反量化（系数乘量化表）、对每个 8×8 块做二维逆 DCT、再加回 128 拼成整幅图像，并按 $PSNR = 10 \cdot \log_{10}(255^2 / MSE)$ 计算重建图相对原图的峰值信噪比。
再把量化表整体放大 4 倍重做一遍编解码，比较两次的 PSNR。
把重建图命名为 R、标准量化表的 PSNR 命名为 PSNR、放大 4 倍的命名为 PSNR4。

例 18.4 运行结果 · 重建图与 PSNR

运行输出如下：

标准量化表 PSNR = 32.81 dB

4倍量化表 PSNR4 = 27.20 dB

Original



Reconstructed (Standard Q)
PSNR = 32.81 dB



Reconstructed (Q × 4)
PSNR = 27.20 dB



例 18.4 核对与易错点

对结果的核验

- PSNR 应为有限值 (JPEG 是有损, > 25 dB 即合格); 量化表放大 4 倍后 PSNR 明显下降, 重建图上 8×8 的块效应方块也更明显, 量化越粗、失真越大都反映在这个数值上。

易错点: PSNR 算成无穷大, 或被饱和截断而虚高

- PSNR 算成 ∞ , 多半是漏了量化、或反量化与量化不对应、重建等于原图; 而 JPEG 有损、PSNR 应是有限值。
- MSE 须在 double 上算: 若直接对 uint8 相减, 负差被饱和截成 0、MSE 偏小、PSNR 虚高, 须先转 double。

§18.3 信息隐藏

- 信息隐藏把秘密信息嵌入一个看似正常的载体，使外人难以察觉其存在（连“是否藏了信息”都难判断），数字水印是常见应用。这一点与压缩正好相反：压缩丢掉人眼不在意的部分，隐藏则利用这些不被注意的部分来嵌入信息。
- 空域 **LSB**：把信息位逐一替换像素亮度的最低有效位（权重 1 的那一位，改动它人眼无法察觉），简单、容量大；缺点是一经有损压缩，所嵌信息即被当作高频丢弃。
- 变换域（**DCT 域**）：在 JPEG 的 8×8 DCT 框架上，把信息嵌在中频系数上。当嵌入扰动大于压缩的量化步长时即能抗压缩。这是变换域比空域稳健的根本原因。
- 例 18.5 空域 LSB、例 18.6 DCT 域 $\pm\Delta$ ，对照隐蔽性与抗压缩性。

例 18.5 空域 LSB 隐写

实现空域最低有效位（LSB）信息隐藏与提取，并检验它的抗 JPEG 压缩能力。

请读入同目录下的清华二校门灰度照片 `thu_gate.jpg`，实现最低有效位（LSB）隐写：

生成一段 0/1 秘密比特（用固定随机种子以便复现），逐个写入前若干个像素的最低位，再从含密图把这些最低位读回。把原图命名为 `I`、秘密比特命名为 `bits`、含密图命名为 `stego`、提取结果命名为 `extracted`，并把原图与含密图并排显示。

例 18.5 运行结果 · 原图与含密图几乎无差别

运行输出如下：

秘密比特提取正确。

原图与含密图肉眼几乎无差别，只改动了权重 1 的最低位。

原图 I



含密图 stego



例 18.5 核对与易错点

对结果的核验

- 提取比特应与嵌入完全一致（误码 0）；含密图与原图逐像素差至多为 1（只改动最低位，差 ≥ 2 即写错了位）；PSNR 极高（约 70 dB），隐蔽性好。但最低位一经 JPEG 压缩就被当作高频丢弃、信息全毁，这是 LSB 的短板，例 18.6 用 DCT 域解决。
- 可用 `mod(double(stego),2)` 独立复提最低位、与 `bits` 比对，不依赖模型自报。

易错点：位序号从 1（最低位）起

- `bitset(x,1,b)` 写最低位、`bitget(x,1)` 读最低位，位序从 1 起、非从 0；写成 0 或 8 会改到别的位，含密图逐像素差就会 ≥ 2 ，可据此判错。原课程用通信工具箱的 `de2bi` / `bi2de`，已改用基础位运算，学生侧无需该工具箱。

例 18.6 DCT 域中频系数 $\pm\Delta$ 隐藏

实现 DCT 域中频系数 $\pm\Delta$ 的信息隐藏与提取，并检验其抗压缩能力。

请读入同目录下的清华二校门灰度照片 `thu_gate.jpg`，实现 DCT 域信息隐藏：把图像按 8×8 分块、对每块做二维 DCT，在每块一个固定中频系数位置嵌入 1 比特——嵌 1 就把该系数置为 $+\Delta$ 、嵌 0 置为 $-\Delta$ ， Δ 取一个明显大于后续量化步长的值（如 30）；提取时对含密图重新分块做 DCT、看该系数符号（ >0 判 1，否则判 0）。再用一个比 Δ 小的步长（如 8）做一次均匀量化、重新提取，验证抗压缩能力。

把原图命名为 `I`、秘密比特命名为 `bits`、含密图命名为 `stego`、直接提取命名为 `extracted`、量化后提取命名为 `extracted2`。

例 18.6 运行结果·含密图与抗压缩

运行输出如下：

直接提取错误比特数：0 / 2665 （误码率：0.0000）

Qstep=8 量化后提取错误比特数：0 / 2665 （误码率：0.0000）

含密图上隐约可见交叉纹理（中频被改动所致），比 LSB 稍易察觉，PSNR 约 35 dB 量级。

原图



含密图



例 18.6 核对与易错点

对结果的核验

- 直接提取误码 0；含密图 PSNR 约 35 dB，比 LSB 略低（中频改动隐约可见）；关键在于用比 Δ 小的步长（ $q = 8 < \Delta = 30$ ）量化后再提取，误码仍为 0。在同样的嵌入信息量下与例 18.5 对照：LSB 更隐蔽（PSNR 更高、几乎不可见）却一经压缩就全毁，DCT 域稍欠隐蔽却扛得住压缩。隐蔽性与抗压缩性是一对取舍。

易错点： Δ 须明显大于量化步长

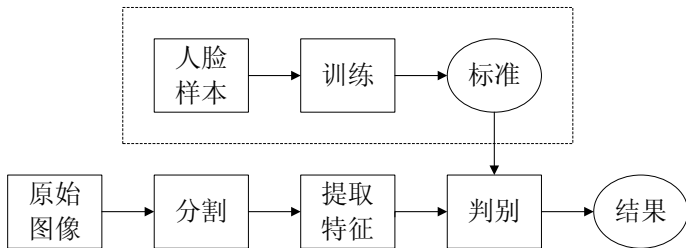
- Δ 必须明显大于压缩的量化步长 q ，符号才不会被量化翻转、提取才不出错；中频位置的选取也很关键，选 DC 或太低频会显著损伤画质，选太高频则一经压缩即被消除。嵌入后须 `idct2` 回空域、限幅 $0 \sim 255$ 再转 `uint8`。

§18.4 人脸检测（一）：思路与颜色特征

- 第 17.4 节用相关在图里找已知目标，相关的核心就是内积：目标与模板逐点相乘再求和。人脸检测是同一思路，只是不逐点比对，而是用一个特征概括一块图，再用内积衡量它与肤色标准特征的吻合程度。
- 特征取颜色：某区域内各颜色出现频率构成的矢量，它实为一个概率分布（各频率之和为 1）。
- 直接用 **RGB** 颜色作特征：把 R、G、B 每个分量量化到 L 位（如 $L = 3$ ，每通道 8 级、共 $8^3 = 512$ 种颜色），统计每种颜色出现的频率。颜色种类减少，既缩短了特征、也更鲁棒；这种 RGB 直方图简单有效，缺点是对光照较敏感。

§18.4 人脸检测（二）：训练标准特征、检测算内积

- 训练：收集人脸相片、统计颜色频率、平均得“肤色标准特征”。
- 检测：算待测区域特征 h 与标准 feat 的相似度。两者都是概率分布，用 **Bhattacharyya** 系数 $\rho = \sum_n \sqrt{h_n \text{feat}_n}$ ，它正是 $\sqrt{h}$ 与 $\sqrt{\text{feat}}$ 的夹角余弦， $\rho \in [0, 1]$ ，接近 1 即判为肤色。



训练得肤色标准特征、检测算相似度

例 18.7 肤色 RGB 颜色直方图特征与相似度判别

我有一批人脸皮肤训练图，文件名是 1.bmp、2.bmp、……、33.bmp（RGB 彩色，放在当前搜索路径，`imread('1.bmp')` 即可读到）。

请用 RGB 颜色直方图作肤色特征：把每张图的 R、G、B 分量各量化到 L 位（取 $L=3$ ，即每通道 8 级、共 $8^3=512$ 种颜色），统计各颜色出现的频率（频率之和为 1）作为该图特征。

把 33 张图的特征平均，得到肤色标准特征 `feat`。

再写一个相似度：对某块特征 `h` 与标准 `feat`，用 Bhattacharyya 系数 $\rho = \text{sum}(\text{sqrt}(h.*\text{feat}))$ （`h` 与 `feat` 都是概率分布、各元素之和为 1）， ρ 越接近 1 越相似。

分别测一块肤色图（如 5.bmp）和一块明显非肤色的纯绿块，比较两者与标准特征的 ρ 。

把标准特征命名为 `feat`、肤色块相似度命名为 `s_skin`、非肤色块相似度命名为 `s_nonskin`。

例 18.7 运行结果与核对

运行输出如下：

```
肤色块 5.bmp 与肤色标准特征的相似度 s_skin    = 0.8396  
非肤色纯绿块与肤色标准特征的相似度 s_nonskin = 0.0000
```

对结果的核验

- feat 应为 512 维 (8^3)、非负、 $\sum = 1$ (一个概率分布)。
- Bhattacharyya 相似度落在 $[0, 1]$ ：肤色块与标准接近、 s_{skin} 明显偏大 (本例 0.84)；纯绿块的颜色与肤色几乎不重叠、 $s_{\text{nonskin}} \approx 0$ 。取一个阈值即可判别是否肤色。

例 18.7 易错点

易错点：直方图须归一化、量化位数别弄错

- 每张图的直方图必须归一化（除以像素总数、使各频率之和为 1），否则大小不同的块无法比较，Bhattacharyya 系数也不再落在 $[0, 1]$ 。
- RGB 量化：把 $0 \sim 255$ 的分量量化到 L 位，是右移 $8 - L$ 位（或除以 2^{8-L} 取整）； L 越小颜色越少、越鲁棒但越粗，本例取 $L = 3$ 。

围绕一张照片（手机自拍或一张清华合影），完成图像在实际中会经历的三种处理：压缩存储、嵌入隐藏信息、检测其中的人脸。三者都与压缩有关：隐藏的信息须经受压缩，检测出的人脸区域则应在压缩中予以保留。压缩与隐藏在照片的灰度版上完成，人脸检测用彩色版。提示词须自行撰写，结果是否正确由专业知识判断。

任务一 JPEG 压缩与画质的权衡 把照片转灰度，用 8×8 分块 DCT + 量化做 JPEG 压缩；用不同粗细的量化表（标准、 $\times 2$ 、 $\times 4$ 、 $\times 8$ ）各压一遍。

- 报告各档的 PSNR 与被量化成 0 的系数比例，核对 PSNR 随量化变粗单调下降；指出从哪一档起 8×8 块效应肉眼可见。不要求实现 Huffman 码流。

综合大作业 任务二 嵌入隐藏信息、检验抗压缩

在灰度照片里嵌入一段信息（一句话，或一个小的 0/1 图案）。先用空域 **LSB**（改最低位）嵌入、提取核对无误。本任务的要点在于：把含密照片送入任务一的 JPEG 压缩，考察压缩之后能否再提取出信息。LSB 嵌在最低位，压缩一旦改动像素，所嵌信息即被破坏。然后改用 **DCT** 域 $\pm\Delta$ 嵌入，令嵌入步长 Δ 大于压缩的量化步长 q ，信息便能经受压缩。

要求

- 报告 LSB 嵌入与提取的误码（应为 0）、含密图与原图逐像素差（应 ≤ 1 ）。
- 把含密图送入任务一不同粗细的压缩，记录提取误码随量化变粗如何上升。
- 用 DCT 域、令 $\Delta > q$ ，核对压缩后误码率是否更低。用自己测的误码率，验证变换域和空域相比谁更抗压缩。

用人脸检测的结果指导压缩。先对彩色照片检测人脸（肤色区域）：按 RGB 颜色是否落在常见肤色范围内逐块判断，得到人脸位置，用现成的经验肤色范围即可、不必训练。再回到压缩：人脸块用细量化保画质、背景块用粗量化重压缩（即 ROI 感兴趣区域编码），与全图均匀压缩作对比。

要求

- 压缩程度用量化后非零系数的总数近似（非零越少、码流越短，无须实现熵编码）。调两种方案的量化档位、使非零系数总数相近，再对比 ROI 与全图均匀压缩：前者人脸区域 PSNR 应明显更高，背景更模糊。
- 诚实分析肤色检测的误检（手、橙黄色物体被判为人脸）与漏检（暗光、侧脸），并指出它如何影响压缩：若把手误判为人脸，便会把有限的码率耗费在其上。

提交材料

- 完整对话记录：本人撰写的每条提示词、模型给出的代码，以及至少一处找错与修正的回合（提示词须本人撰写、非照抄正文）。
- 验证报告：每项任务写明所用判据、核对过程（数值或图）与结论。
- 程序与图像：可运行程序、本人的原始照片与各步结果图。

评分 不以“代码是否写出”计（提示词自撰、参考程序随书提供），初步考虑评价原则：系统完成度 25%、验证报告 35%、对话与找错 25%、拓展与趣味 15%。